AIMLCZG557

Complete Slide Exercise and Practice Question Bank

Topic-wise inventory, difficulty tags, beginner time estimates, original question slides, and fully worked solutions.

Coverage: all distinct end-of-deck lecture exercises and webinar practice questions found in the local ACI slide collection. Exact duplicate deck copies are consolidated.

15 distinct question sets | 47 answerable parts | 8.5-11.5 beginner hours

How to use this packet

• Attempt the original question slides first. The solution section begins only after all questions.

• For numerical questions, write g(n), h(n), f(n), OPEN and CLOSED explicitly.

• The estimated time assumes the topic has been studied once. First-time learning will take longer.

• Difficulty is relative to a beginner, not an official instructor rating.

Counting convention

A question set is one exercise/scenario. An answerable part is each independently requested task. Exact duplicates across CS05, CS06 and source-bundle copies were counted once.

Difficulty distribution

Tag	Meaning for a beginner	Sets	Parts
Easy	Direct recall or one short calculation.	3	4
Medium	Several definitions or a short algorithm trace.	6	19
Hard	Multi-stage reasoning, tables, or implementation.	5	21
Very Hard	Long numerical with ambiguous/nonstandard costs.	1	3

Topic-wise workload

Topic group	Sets	Parts	Estimated beginner time
PEAS and task environments	3	12	60-85 min
Problem formulation	1	3	35-50 min
A*, GBFS and heuristics	6	23	4 hr 15 min - 6 hr
Local search and hill climbing	2	4	1 hr 35 min - 2 hr 40 min
Genetic algorithms	1	2	10-15 min
Minimax and static evaluation	2	3	45-70 min
TOTAL	15	47	About 8.5-11.5 hours

Complete question inventory

#	Topic	Question set	Parts	Difficulty	Beginner time
1	PEAS	Grammarly interactive tutor	1	Easy	10-15 min
2	Agents and environments	Path-finding robot	7	Medium	25-35 min
3	Environment classification	15-puzzle, poker, diagnosis, taxi	4	Medium	25-35 min
4	Problem formulation	Drone, maze and patient scheduling	3	Medium	35-50 min
5	A* search	S-G directed weighted graph	1	Medium	20-30 min
6	GBFS, A* and heuristics	Weighted graph from B to G	5	Hard	45-60 min
7	Heuristic design	Warehouse packages, keys and relaxations	8	Hard	50-70 min
8	A* search	Warehouse grid with rewards and hazards	3	Very Hard	60-90 min
9	A* search	Flood-rescue grid	3	Hard	50-70 min
10	Problem formulation and PEAS	Autonomous supermarket checkout	3	Medium	30-45 min
11	Local beam search	Select the next beam	1	Easy	5-10 min
12	Hill climbing	Random restart, annealing and GA extensions	3	Hard	90-150 min
13	Genetic algorithms	Crossover and selection	2	Easy	10-15 min
14	Minimax	Numerical game tree	1	Medium	15-25 min
15	Minimax and evaluation	Castle Battle	2	Hard	30-45 min

Questions

The following pages preserve the original slide wording, diagrams, grids and tables. Solutions are intentionally placed after the complete question section.

Recommended order for a beginner

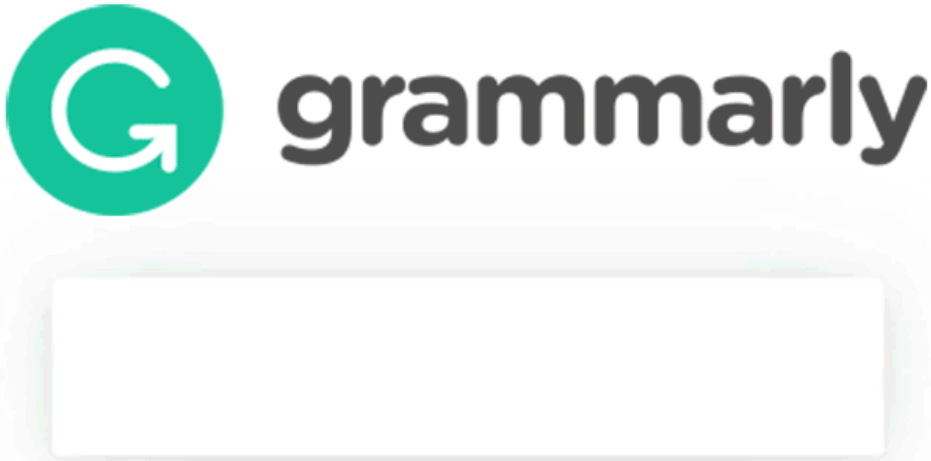
- Start with Questions 1, 3, 11 and 13.
- Then attempt Questions 2, 4, 5, 10 and 14.
- Finish with Questions 6, 7, 9, 15 and finally Question 8.

1

Intelligent Agents, PEAS & Task Environments

Exercises on PEAS specification and environment classification.

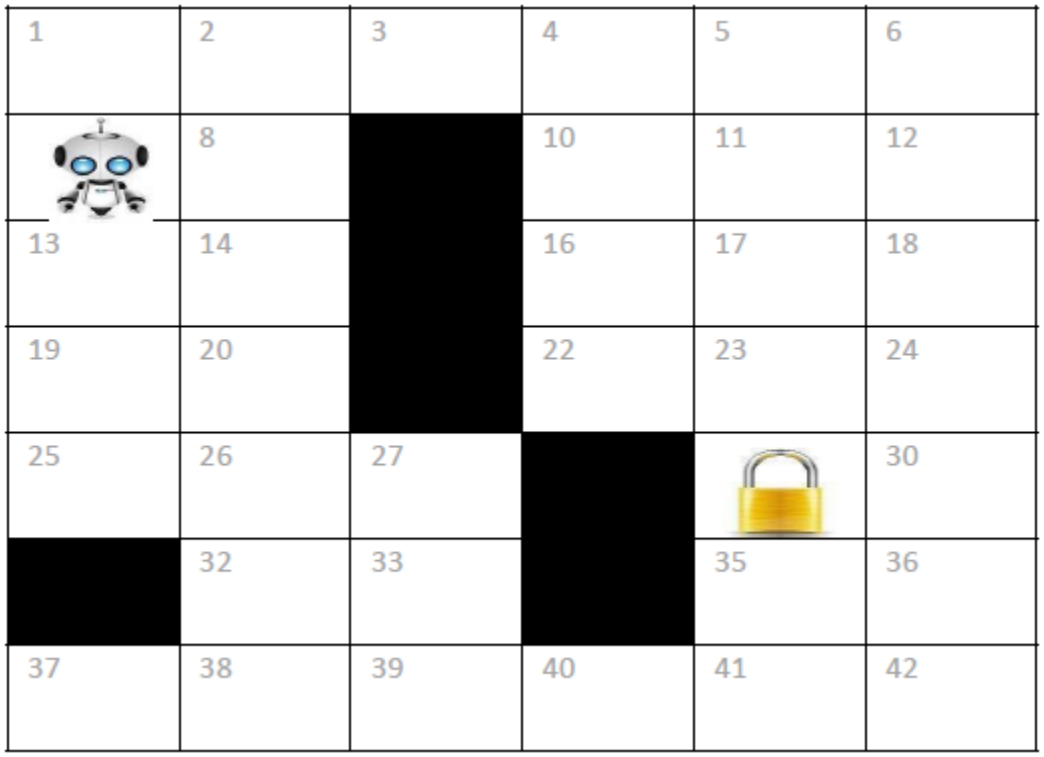
Exercise 1: Write the PEAS for an Interactive English tutor such as Grammarly



- Performance measure: ?
- Environment: ?
- Actuators: ?
- Sensors: ?

Exercise 2: Path Finding Robot

PEAS for this problem?



Observability?

Number of Agents?

Determinism?

Dynamicity?

Episodic or Sequential?

No of States?

Exercise 3: Identify the Environment

- 15-Puzzle game
- Poker
- Medical Diagnosis
- Taxi Driving

2

Problem Formulation

Formulate initial state, actions, transition model, goal test and path cost.

Exercise 1

innovate achieve lead

For the below tasks:

1. Planning a trip with a delivery drone
2. Solving a maze
3. Scheduling patient appointments in a clinic

Formulae **the problem** by specifying:

- Initial state
- Possible actions
- Transition model (effects of actions)
- Goal test
- Path cost (optional)

Think about:

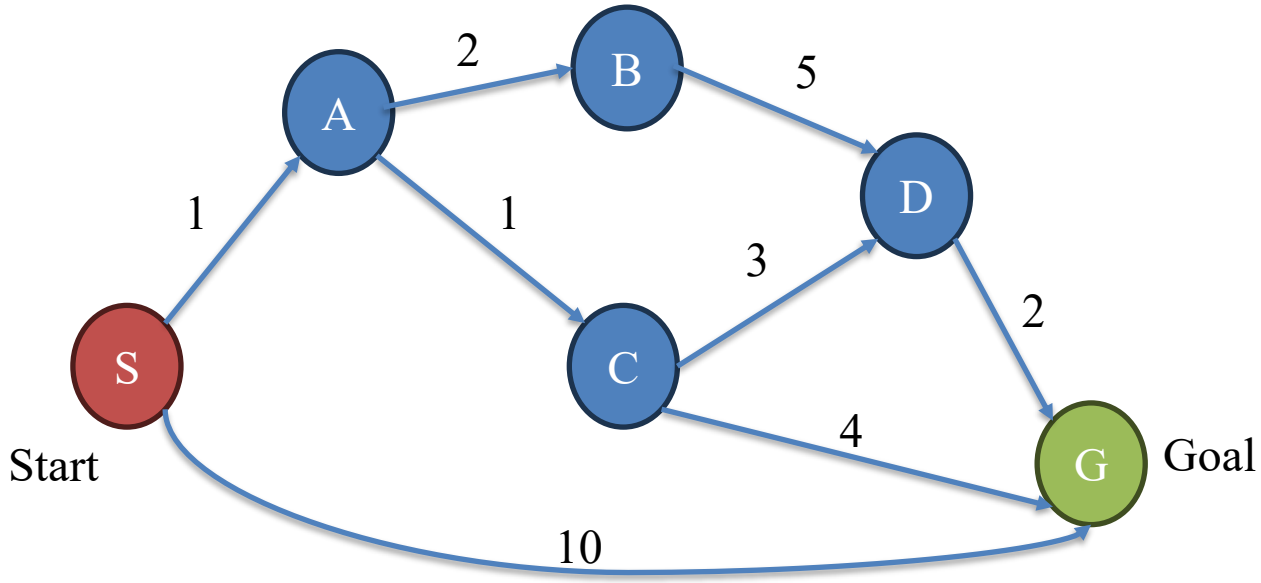
- Any **missing components**
- Assess **well-formulated-ness** (Does it meet criteria for a well-formed problem?)

3

Informed Search, A* & Heuristic Design

A*, GBFS, admissibility, consistency, relaxed problems and exam-style grid searches.

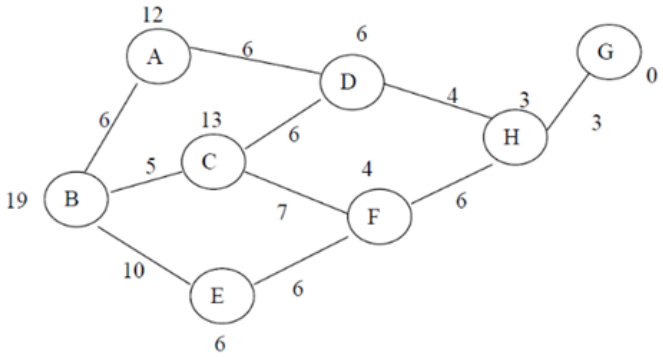
Exercise 2



Apply A* algorithm on the above graph, given the heuristics: h(S)=5, h(A)=3, h(B)=4, h(C)=2, h(D)=6 and h(G) = 0

Exercise 1

- Consider the following weighted graph. The number beside each node represents its heuristic value $h(n)$,which estimates the cost from that node to the goal node G.
- The start node is B, and the goal node is G.



- Apply GBFS on this graph.
- Apply A* Algorithm on this graph.
- Compare your findings from the above two
- Check if the heuristics is admissible and consistent.

Exercise 2

- A warehouse robot has to collect three packages and reach the exit.
- The warehouse has rooms connected by corridors. Some corridors have locked doors. The robot can move from one room to an adjacent room with cost 1. To pass through a locked door, the robot must first collect the matching key.
- **Warehouse layout** Rooms:

$$S, A, B, C, D, E, F, G, X$$

where:

- S= start room
- X= exit room
- Packages are located at C, F, and G
- Key $K_1$is located at B
- Key $K_2$is located at E

The robot must collect all three packages at C, F, and G, then reach X.

Exercise 2 Continued

Corridor	Cost	Condition
(S-A)	1	Open
(A-B)	1	Open
(A-C)	2	Open
(B-D)	2	Requires (K1)
(C-D)	2	Open
(D-E)	1	Open
(E-F)	2	Requires (K2)
(D-G)	3	Open
(G-X)	2	Open
(F-X)	2	Open

Part A: State Representation

Define a state representation for this problem.
Your state should include:
1.Current room of the robot.
2.Packages already collected.
3.Keys already collected.
Write the initial state and one possible goal state.

Exercise 2 Continued

Part B: Relaxed Problems

A relaxed problem is obtained by removing one or more constraints from the original problem.

Consider the following relaxed versions of the warehouse problem:

- **Relaxation 1:** Ignore all locked-door constraints. The robot can pass through any corridor.
- **Relaxation 2:** Ignore package collection order. Estimate the cost only as the shortest distance from the current room to the exit.
- **Relaxation 3:** Assume the robot can collect any remaining package instantly once it enters the same connected component, but it must still move to the exit.
- **Relaxation 4:** Ignore the robot's current location and estimate only the minimum cost needed to connect all uncollected package locations and the exit.

Answer the following:

1. For each relaxed problem, explain which constraint has been removed.
2. Which relaxed problem is likely to give the weakest heuristic?
3. Which relaxed problem is likely to give the strongest heuristic?
4. Why does the solution cost of a relaxed problem give an admissible heuristic for the original problem?
5. Construct few heuristics by picking some relaxed version of the problem.

Exercise 1

A warehouse robot begins at C1, tasked with collecting Items (It) and reaching the Dock (DV) at C4 on a 4x4 grid. The robot must navigate hazards like blocked cells (#), Spills (SP) with high movement costs, while leveraging Energy packs (E) to reduce costs. Using the A* algorithm, the robot calculates the optimal path by considering both actual path cost and a heuristic. The agent has four distinct actions: Move Up, Move Down, Move Left, and Move Right. However, it cannot move into blocked cells. The warehouse is represented as a 4×4 grid below:

	1	2	3	4
A	It	E		SP
B	SP	It	E	SP
C	S	E	It	DV
D	It	SP	SP	E

Entering SP: +12; collecting It: −8 (reward); energy pack E: +1.
Heuristic h(n) = Manhattan(robot, DV) + (# remaining Items) + (# adjacent SP to robot in resultant state).

a) Expand and depict the search tree for exactly 2 levels. (Level 0 = Start).
b) Calculate Path costs and heuristics (g/h/f) for all generated nodes.
c) Apply A* search and show all the steps.

79

Question 1:

A Rescue Agent is deployed in a flood-affected city to deliver medicines from the base camp at (X1, Y1) to a relief shelter located at (X5, Y4). The agent must efficiently plan the route to ensure the timely delivery of the medical supplies. The city is represented as a 5x5 grid. The agent is equipped with four distinct actions: Move Up, Move Down, Move Left, and Move Right. However, it cannot move into waterlogged and blocked cells. The city is represented as a 5×5 grid below:

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Legend:

S = Start (Base Camp)

G = Goal (Relief Shelter)

N = Normal cell (cost +1)

W = Waterlogged (+5 cost)

E = Energy pack (–2 cost)

B = Blocked (not traversable)

Heuristic:

H(n)=Manhattan Distance to Goal +
No. of Waterlogged cells adjacent to position

For the above problem do the following:

a) Expand and depict the search tree for exactly 3 levels. (Level 0 = Start).

b) Calculate path cost and heuristic values for all nodes at Level 1–3.

c) Apply A* search and show all steps.

Manhattan distance = For two points (P(x1, y1)) and (Q(x2, y2)), the formula is:
D=|x1-x2|+|y1-y2|)

PEAS and Environment example

Question 2:

A large supermarket chain wants to implement an Autonomous Checkout System where customers can pick items and walk out without scanning them manually. The system uses cameras, weight sensors, and RFID scanners to track selected items, update the bill, and charge customers automatically through a mobile app.

a. Provide the complete problem formulation.

b. Provide the PEAS description.

c. Identify the various dimensions of the task environment with appropriate justification for each.

4

Local Search & Hill Climbing

Local beam selection and extension tasks for hill-climbing code.

Exercise 1

- Suppose local beam search has k = 3 and the following current states:

Current State	Heuristic Value
S1	12
S2	15
S3	10

- Their successors are:

Successor	Heuristic Value
A	16
B	11
C	14
D	18
E	13
F	17

If the goal is to maximize, which three states will be selected for the next beam?

Hill Climbing - Food for Thought

- Add code to convert stochastic hill climbing into random-restart hill climbing.
- Solve the travel-optimization problem using simulated annealing.
- Implement the solution for the same problem using a genetic algorithm.

5

Genetic Algorithms

Single-point crossover and fitness-based selection.

Exercise 2

1. Given two parent chromosomes:

- Parent 1: [1, 2, 3, 4, 5, 6, 7, 8]
  Parent 2: [8, 7, 6, 5, 4, 3, 2, 1]
- Using single-point crossover after the 4th position, generate two children.

2. Suppose the fitness values of four individuals are:

Individual	Fitness
A	10
B	30
C	40
D	20

Which individual has the highest chance of being selected? Why?

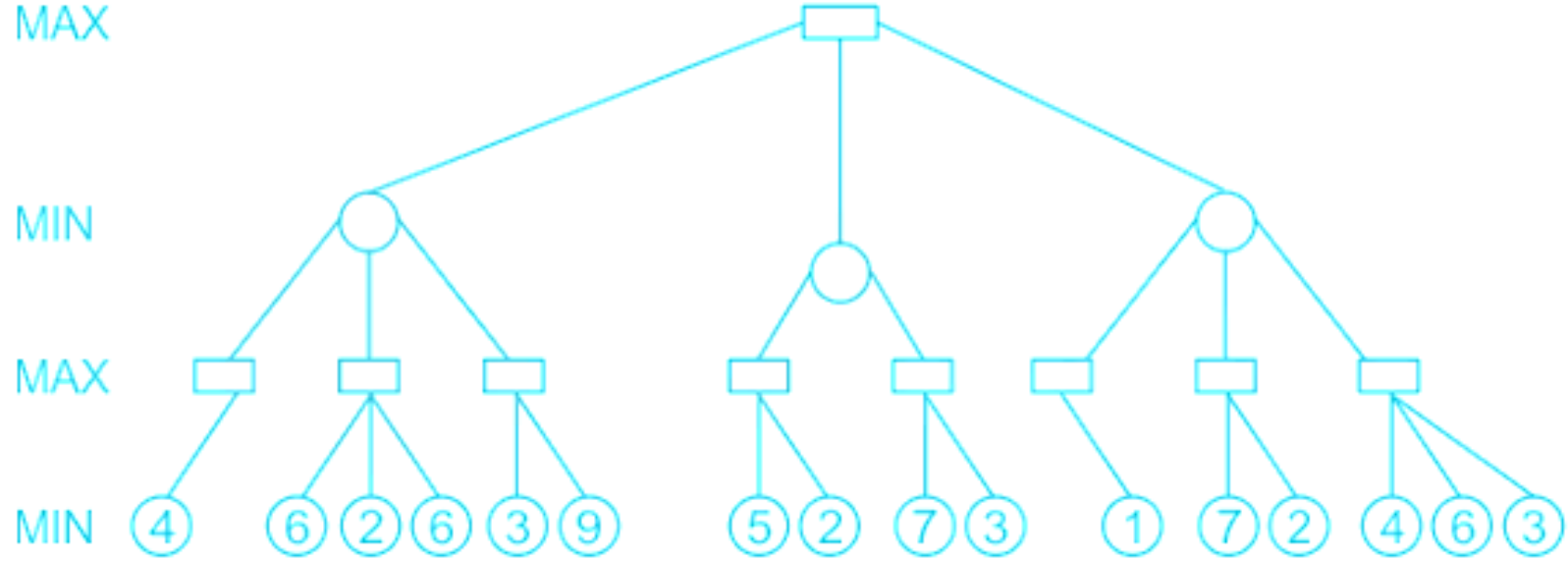
6

Adversarial Search, Minimax & Static Evaluation

Minimax tree backup and a two-ply castle-battle evaluation problem.

Exercise 1

Consider the following minimax game tree search



What will be the value propagated at the root?

Exercise 2: Castle Battle Game

- Two players are playing a small strategy game.
- **MAX** is the attacking army.
- **MIN** is the defending army.
- MAX wants to capture the castle.
- MIN wants to prevent MAX from capturing the castle.
- The game is not fully played until the end. We will search only **2 moves ahead** and then use a **static evaluation function**.

Part A: Draw the Game Tree

At the current position, MAX has three possible moves:

1. **Attack Left Gate**
2. **Attack Main Gate**
3. **Attack Right Gate**

After each MAX move, MIN has two possible replies.

MAX move	MIN reply 1	MIN reply 2
Attack Left Gate	Reinforce Left Gate	Counterattack Left
Attack Main Gate	Reinforce Main Gate	Block Supply Route
Attack Right Gate	Reinforce Right Gate	Counterattack Right

32

Exercise 2: Castle Battle Game

Task 1

- Draw the game tree up to depth 2.
- The root node should be MAX's turn.
- The second level should be MIN's turn.
- The leaf nodes should be the board positions after MIN replies.

Part B: Design a Static Evaluation Function

At each leaf node, the game is not over. So, we need to estimate how good the position is for MAX.

Assume we judge the position using four features:

Feature	Meaning
Soldier advantage	MAX soldiers − MIN soldiers
Gate damage	Damage done to castle gates
MAX safety	How safe MAX's army is
Supply control	Whether MAX controls the supply route

$$\text{Evaluation} = 2 \times \text{Soldier Advantage} + 3 \times \text{Gate Damage} + 2 \times \text{MAX Safety} + 4 \times \text{Supply Control}$$

Exercise 2: Castle Battle Game

Leaf	Soldier Advantage	Gate Damage	MAX Safety	Supply Control
L1	2	3	2	0
L2	1	4	0	1
M1	3	2	3	0
M2	1	3	1	2
R1	2	2	2	1
R2	0	5	0	1

Given the above feature values (leaf), compute the evaluation scores using the eval function.

Fully Worked Solutions

Symbols used in search solutions

Symbol	Meaning
g(n)	Actual path cost from the start state to node n.
h(n)	Estimated remaining cost from node n to a goal.
f(n)	A* evaluation value: f(n) = g(n) + h(n).
OPEN	Generated nodes waiting to be selected.
CLOSED	Nodes already expanded.

Tie-breaking can change the expansion order without changing the final optimal path when the heuristic and costs satisfy A* assumptions.

Solution 1: PEAS for an interactive English tutor

Difficulty: Easy | Beginner time: 10-15 minutes

PEAS component	Model answer
Performance measure	Correct grammar and spelling; useful explanations; few false corrections; fast response; improvement in learner writing; privacy and accessibility.
Environment	Learner, typed document, browser/editor, language rules, writing context, dictionaries and style guides.
Actuators	Underline/highlight errors, replacement suggestions, explanations, score/feedback, notifications and accepted edits.
Sensors	Keystrokes, document text, cursor position, accepted/rejected suggestions, selected language, writing goals and user settings.

Exam-ready sentence: The tutor is rational when it selects feedback expected to improve writing quality while minimizing incorrect or distracting suggestions.

Solution 2: Path-finding robot

Difficulty: Medium | Beginner time: 25-35 minutes

Assumption: the map is known, the obstacles are fixed, movement is one cell at a time, and the lock cell is the destination. The picture has 42 physical cells and 6 blocked cells.

Dimension	Classification	Justification
Observability	Fully observable	The robot is assumed to know its cell, goal and obstacle map.
Number of agents	Single-agent	Only the robot is choosing actions.
Determinism	Deterministic	A legal move has a predictable resulting cell.
Dynamicity	Static	The obstacles and goal do not move while planning.
Episodic/sequential	Sequential	Each move changes the next available actions and path cost.
State type	Discrete	Positions and moves are finite grid cells/actions.
Number of states	36 traversable position states	42 cells minus 6 black blocked cells. If lock/key status were modeled, the state count would increase.

PEAS

Performance	Environment	Actuators	Sensors
Reach the lock/goal safely; minimum moves; time, energy, and collisions.	6x7 grid, free cells, blocked cells, start and lock/goal.	Move up, down, left and right; stop.	Position sensor, obstacle/map sensor, goal detector and movement feedback.

Solution 3: Classify four task environments

Difficulty: Medium | Beginner time: 25-35 minutes

Environment	Observable	Agents	Determinism	Sequence	Dynamicity	State type	Known?
15-puzzle	Full	Single	Deterministic	Sequential	Static	Discrete	Known
Poker	Partial	Multi	Stochastic	Sequential	Static/semi-dynamic	Discrete	Known rules
Medical diagnosis	Partial	Usually single decision-maker	Stochastic	Sequential	Dynamic	Hybrid	Partly known
Taxi driving	Partial	Multi	Stochastic	Sequential	Dynamic	Continuous	Partly known

Key justifications

- **15-puzzle:** every tile is visible and each legal move has a known result.
- **Poker:** opponents' cards are hidden and card dealing introduces chance.
- **Diagnosis:** the true disease state is hidden; tests provide imperfect evidence and patient state may change.
- **Taxi:** other road users act independently and the world changes during deliberation.

Solution 4: Problem formulation for three scenarios

Difficulty: Medium | Beginner time: 35-50 minutes

Component	Delivery drone	Maze	Patient scheduling
Initial state	Drone at depot; deliveries unserved; initial battery/load.	Agent at maze entrance.	Unassigned patients, clinicians and available slots.
State	Location, delivered set, battery, load and time.	Current cell (and visited set for graph search).	Partial appointment assignment plus remaining capacity.
Actions	Fly to adjacent waypoint, deliver, recharge or wait.	Move to an adjacent open cell.	Assign/reschedule/cancel an appointment.
Transition	Update location, energy, time, load and delivered set.	Move to selected legal neighboring cell.	Reserve/release slot and update constraints.
Goal test	All required deliveries complete and safety condition satisfied.	Current cell is the exit.	Every required patient has one feasible appointment.
Path cost	Distance/time/energy/risk; use a weighted sum if needed.	Number of steps or weighted terrain cost.	Waiting time, overtime, clashes, unfairness and unused capacity.
Well-formed?	Yes after weather, battery and no-fly rules are specified.	Yes if walls, moves and costs are explicit.	Yes if clinician, duration, priority and resource constraints are explicit.

A well-formed problem has a precise initial state, finite/legal actions, a transition model, a testable goal and a comparable path-cost function.

Solution 5: A* on the directed S-G graph

Difficulty: Medium | Beginner time: 20-30 minutes

Heuristics: h(S)=5, h(A)=3, h(B)=4, h(C)=2, h(D)=6, h(G)=0.

Step	Selected	Generated/updated nodes	OPEN after expansion
0	S: g=0, h=5, f=5	A: (1,3,4); G: (10,0,10)	A(4), G(10)
1	A: g=1, h=3, f=4	B: (3,4,7); C: (2,2,4)	C(4), B(7), G(10)
2	C: g=2, h=2, f=4	D: (5,6,11); G improved to (6,0,6)	G(6), B(7), D(11)
3	G: g=6, h=0, f=6	Goal reached	-

Final path: S -> A -> C -> G. **Total cost:** 1 + 1 + 4 = 6.

Solution 6: GBFS, A* and heuristic quality

Difficulty: Hard | Beginner time: 45-60 minutes

Graph facts: h(A)=12, h(B)=19, h(C)=13, h(D)=6, h(E)=6, h(F)=4, h(H)=3, h(G)=0.

Greedy Best-First Search

GBFS selects only the smallest h value. Expansion: B -> E -> F -> H -> G. Path cost = 10 + 6 + 6 + 3 = **25**.

A* search

Selected	Important OPEN entries as (g,h,f)
B	E(10,6,16), A(6,12,18), C(5,13,18)
E	A(6,12,18), C(5,13,18), F(16,4,20)
A/C tie	Expanding C gives D(11,6,17). Expanding A first temporarily gives D(12,6,18), later improved through C.
D	H(15,3,18)
H	G(18,0,18)
G	Goal reached

A* path: B -> C -> D -> H -> G. **Cost:** 5 + 6 + 4 + 3 = 18.

Comparison

- GBFS is faster here but returns cost 25 because it ignores cost already paid.
- A* combines g and h and finds the true shortest path of cost 18.

Admissibility and consistency

Node	True remaining cost h*(n)	Given h(n)	Admissible?
A	13	12	Yes
B	18	19	No
C	13	13	Yes

Node	True remaining cost h*(n)	Given h(n)	Admissible?
D	7	6	Yes
E	15	6	Yes
F	9	4	Yes
H	3	3	Yes
G	0	0	Yes

The heuristic set is **not admissible** because h(B)=19 > h*(B)=18. It is also **not consistent**: on B-C, 19 <= 5+13 is false; on B-A, 19 <= 6+12 is also false.

Solution 7: Warehouse state representation and relaxed heuristics

Difficulty: Hard | Beginner time: 50-70 minutes

Part A: state representation

Use state **(r, P, K)**, where r is the current room, P is the set of collected packages, and K is the set of collected keys.

- **Initial:** (S, empty set, empty set).
- **Goal:** (X, {C,F,G}, K), where K contains any keys acquired along the legal route, normally {K1,K2}.
- A move along corridor (u,v) is legal if its required key is already in K. Entering B adds K1; entering E adds K2; entering C/F/G adds that package.

Part B: removed constraints

Relaxation	Removed constraint	Effect
R1	All locked-door requirements	Still requires package collection and exit, but keys are unnecessary.
R2	All package requirements/order	Only shortest distance from current room to X remains.
R3	Exact travel to each remaining package	Packages in a connected component are treated as collected instantly; movement to X remains.
R4	Robot's current location	Lower bound connects remaining package rooms and X without paying to reach that network.

- **Weakest:** R2, because it ignores every package and can be much smaller than the real remaining work.
- **Strongest likely relaxation:** R1, because it removes only the lock constraint and preserves the most original requirements. R4 can still be a strong, cheap MST-style lower bound.
- **Why admissible:** removing constraints cannot make the relaxed optimum more expensive. Therefore h(n)=optimal relaxed cost <= optimal original remaining cost.

Example admissible heuristics

- h1 = shortest distance from current room to X, ignoring locks and packages.
- h2 = minimum spanning-tree cost over current room, all uncollected package rooms and X, ignoring locks.
- h3 = shortest route that visits all uncollected package rooms and X while ignoring locked-door requirements.

Among admissible heuristics, a larger value that still never overestimates is generally more informative and causes A* to expand fewer nodes.

Solution 8: Warehouse-grid A* with rewards and hazards

Difficulty: Very Hard | Beginner time: 60-90 minutes

Explicit interpretation used below: an item reward of -8 is collected once; revisiting that cell costs +1. Energy, ordinary and dock cells cost +1; spill cells cost +12. A state is (position, collected-items).

Part A/B: exactly two levels

Level	State reached	Collected items	g	h	f
0	C1	-	0	3 Manhattan + 4 remaining + 1 adjacent spill = 8	8
1	B1	-	12	4 + 4 + 0 = 8	20
1	D1	D1	-8	4 + 3 + 1 = 8	0
1	C2	-	1	2 + 4 + 1 = 7	8
2 via B1	A1	A1	4	5 + 3 + 1 = 9	13
2 via B1	B2	B2	4	3 + 3 + 1 = 7	11
2 via D1	D2	D1	4	3 + 3 + 1 = 7	11
2 via C2	B2	B2	-7	3 + 3 + 1 = 7	0
2 via C2	D2	-	13	3 + 4 + 1 = 8	21
2 via C2	C3	C3	-7	1 + 3 + 1 = 5	-2

Part C: classroom-style A* trace

A graph-search implementation using the stated f=g+h ordering and stopping when the first complete goal is removed from OPEN can expand:

```
C1(0,8,8) -> D1(-8,8,0) -> C1[-D1](-7,7,0)
-> C2[-D1](-6,6,0) -> C3[-C3,D1](-14,4,-10)
-> C4[-C3,D1](-13,3,-10) -> B2[-B2,D1](-14,6,-8)
-> B3[-C3,D1](-13,5,-8) -> B2[-B2,C3,D1](-21,5,-16)
-> ... -> A1[all](-28,6,-22) -> ... -> C4[all](-23,1,-22)
```

One first-goal route returned by that trace is C1 -> D1 -> C1 -> C2 -> C3 -> B3 -> B2 -> A2 -> A1 -> A2 -> B2 -> C2 -> C3 -> C4, with cost **-23**.

Mathematically correct caveat and true optimum

This prompt violates ordinary A* assumptions: it has negative edge costs from item rewards, and h(C4)=1 rather than 0 because C4 is adjacent to spill B4. Therefore stopping at the first goal is not guaranteed optimal.

Under the explicit one-time reward model, exhaustive shortest-path calculation gives the better route:

C1 -> D1 -> C1 -> C2 -> B2 -> A2 -> A1 -> A2 -> B2 -> B3 -> C3 -> C4

Cost = -8 + 1 + 1 - 8 + 1 - 8 + 1 + 1 + 1 - 8 + 1 = **-25**.

To make this a standard A* exercise, all transition costs should be nonnegative and the heuristic should be redefined so h(goal)=0. Rewards can instead be tracked as a separate objective or converted to nonnegative penalties.

Solution 9: Flood-rescue grid A*

Difficulty: Hard | Beginner time: 50-70 minutes

The webinar's worked convention treats both W and B cells as non-traversable, even though the legend also displays a W cost. The solution below follows the webinar convention.

Three-level search tree

```
Level 0: (X1,Y1)
  Level 1: Down -> (X2,Y1); Right -> (X1,Y2)
    Level 2 from (X2,Y1): Down -> (X3,Y1)
    Level 2 from (X1,Y2): no new valid child
      Level 3 from (X3,Y1): Right -> (X3,Y2)
```

Node	Level	g	Manhattan	Adjacent W	h	f
(X2,Y1)	1	1	6	1	7	8
(X1,Y2)	1	1	6	2	8	9
(X3,Y1)	2	2	5	0	5	7
(X3,Y2)	3	3	4	2	6	9

A* continuation

Expansion	Chosen node	Reason/result
1	(X1,Y1)	Generate (X2,Y1) f=8 and (X1,Y2) f=9.
2	(X2,Y1)	Generate (X3,Y1) f=7.
3	(X3,Y1)	Generate (X3,Y2) f=9.
4	(X3,Y2)	Continue toward (X4,Y2).
5	(X4,Y2)	Continue to (X5,Y2).
6	(X5,Y2)	Enter energy cell (X5,Y3): g changes from 5 to 3.
7	(X5,Y3)	Generate goal (X5,Y4), final g=4.

Final path: (X1,Y1) -> (X2,Y1) -> (X3,Y1) -> (X3,Y2) -> (X4,Y2) -> (X5,Y2) -> (X5,Y3) -> (X5,Y4). **Total cost:** 4.

Solution 10: Autonomous supermarket checkout

Difficulty: Medium | Beginner time: 30-45 minutes

Problem formulation

Component	Answer
Initial state	Authenticated customer at entrance; empty virtual cart; bill 0; known inventory; linked payment method.
State	Customer/location, detected items, returned items, shelf inventory, sensor evidence, bill, payment and exit status.
Actions	Enter, move, pick/return item, detect event, update cart/bill, verify exit, charge, update inventory, issue receipt or alert.
Transition model	Sensor-confirmed pick adds item and price; return removes it; exit triggers verification, payment, receipt and stock update.
Goal test	Customer exited; selected items correctly identified; bill accurate; payment successful; receipt and inventory update complete.
Path cost	Weighted billing errors, delay, sensor uncertainty, fraud risk, payment failure, false alerts and customer inconvenience.

PEAS

Performance	Environment	Actuators	Sensors
Accurate detection/billing, low delay/theft/false alerts, successful payment, correct inventory, satisfaction.	Aisles, shelves, products, customers, carts, gates, app, payment gateway and inventory system.	Bill/app updates, payment trigger, receipt, inventory update, gate control and staff/security alerts.	Cameras, shelf weights, RFID, entry/exit sensors, app identity, payment and inventory data.

Environment classification

Dimension	Class	Justification
Observability	Partially observable	Sensors may miss or misidentify actions.
Agents	Multi-agent	Many customers, staff and systems interact.
Determinism	Stochastic	Human behavior and sensor errors introduce uncertainty.
Episodic/sequential	Sequential	Earlier detections affect the final bill.
Static/dynamic	Dynamic	Customers and items continue moving.
Discrete/continuous	Hybrid	Items are discrete; tracking signals and movement are continuous.
Known/unknown	Mostly known	Rules are known, but behavior and sensor evidence are uncertain.

Solution 11: Local beam search

Difficulty: Easy | Beginner time: 5-10 minutes

Local beam search with k=3 keeps the three best successors across the whole beam. Because the objective is maximization, sort successor values in descending order:

Rank	Successor	Heuristic value	Selected?
1	D	18	Yes
2	F	17	Yes
3	A	16	Yes
4	C	14	No
5	E	13	No
6	B	11	No

Next beam: {D, F, A}.

Solution 12: Hill-climbing extensions

Difficulty: Hard | Beginner time: 90-150 minutes

The original objective vector is [time, speed, fuel, distance], with cost = abs(time + speed + fuel - distance). Lower cost is better.

A. Random-restart hill climbing

```
def random_restart(restarts, steps):
    best = None
    for _ in range(restarts):
        current = random_solution()
        for _ in range(steps):
            candidate = random_neighbor(current)
            if cost(candidate) < cost(current):
                current = candidate
        if best is None or cost(current) < cost(best):
            best = current
    return best
```

Each restart escapes a local optimum by beginning from a new random state. Keep the best final state over all restarts.

B. Simulated annealing

```
current = random_solution()
T = T0
while T > Tmin:
    candidate = random_neighbor(current)
    delta = cost(candidate) - cost(current)
    if delta < 0 or random() < exp(-delta / T):
        current = candidate
    T = alpha * T       # 0 < alpha < 1
return current
```

A worse candidate can be accepted with probability exp(-delta/T). Early exploration is high; later behavior becomes greedy as temperature T falls.

C. Genetic algorithm

```
population = [random_solution() for _ in range(N)]
repeat for each generation:
    fitness(x) = 1 / (1 + cost(x))
    parents = tournament_or_roulette_selection(population)
    children = arithmetic_or_one_point_crossover(parents)
    mutate one attribute with small probability
    repair values to legal ranges
    population = elitist_replacement(population, children)
return individual with minimum cost
```

Implementation checks

For a sound implementation, speed should have a desired range rather than being minimized blindly, and generated distance should satisfy physical constraints such as distance <= speed x time.

- Keep time, speed, fuel and distance within explicitly declared legal ranges.
- Use a neighborhood operator that makes a small change to one or two attributes.
- Record the best-so-far solution separately so a later random move cannot erase it.
- Stop by target cost, maximum iterations/generations, or no improvement over a fixed window.
- For fair comparison, run all three algorithms with the same cost function and report best cost, iterations and runtime.

Solution 13: Genetic-algorithm crossover and selection

Difficulty: Easy | Beginner time: 10-15 minutes

Single-point crossover after position 4:

```
Parent 1: [1, 2, 3, 4 | 5, 6, 7, 8]
Parent 2: [8, 7, 6, 5 | 4, 3, 2, 1]
Child 1 : [1, 2, 3, 4 | 4, 3, 2, 1]
Child 2 : [8, 7, 6, 5 | 5, 6, 7, 8]
```

If larger fitness is better, individual C has the greatest selection chance because its fitness is 40, the largest value.

Under roulette-wheel selection, total fitness = 10+30+40+20 = 100, so probabilities are A=0.10, B=0.30, C=0.40 and D=0.20.

Solution 14: Minimax value propagation

Difficulty: Medium | Beginner time: 15-25 minutes

Evaluate from the leaves upward. Each square MAX node chooses its largest child; each circular MIN node chooses its smallest child.

Branch	MAX child values	MIN branch value
Left MIN	max(4)=4; max(6,2,6)=6; max(3,9)=9	min(4,6,9)=4
Middle MIN	max(5,2)=5; max(7,3)=7	min(5,7)=5
Right MIN	max(1)=1; max(7,2)=7; max(4,6,3)=6	min(1,7,6)=1

The root is MAX, so its value is max(4,5,1) = **5**. MAX chooses the middle branch.

Solution 15: Castle Battle minimax and static evaluation

Difficulty: Hard | Beginner time: 30-45 minutes

Evaluation = 2(Soldier Advantage) + 3(Gate Damage) + 2(MAX Safety) + 4(Supply Control).

Leaf	Calculation	Score
L1	2(2)+3(3)+2(2)+4(0)	17
L2	2(1)+3(4)+2(0)+4(1)	18
M1	2(3)+3(2)+2(3)+4(0)	18
M2	2(1)+3(3)+2(1)+4(2)	21
R1	2(2)+3(2)+2(2)+4(1)	18
R2	2(0)+3(5)+2(0)+4(1)	19

MIN chooses the smaller score after each MAX attack:

- Attack Left Gate: min(17,18) = 17.
- Attack Main Gate: min(18,21) = 18.
- Attack Right Gate: min(18,19) = 18.

MAX chooses max(17,18,18) = **18**. Attack Main Gate and Attack Right Gate are tied under this evaluation. A deterministic tie-breaker may choose the first, Attack Main Gate.

Final revision checklist

- Can you write PEAS using exact headings without looking?
- Can you classify observability, agents, determinism, sequentiality, dynamicity and discreteness with one justification each?
- Can you maintain OPEN/CLOSED and calculate g, h and f without skipping a step?
- Can you test admissibility against true remaining cost and consistency on every edge?
- Can you propagate minimax values from leaves to root while alternating MAX and MIN?

Mastery reminder: reading these solutions is not evidence of mastery. Attempt at least one PEAS problem, one A* numerical and one minimax tree closed-book.